

Instalación y configuración de servicios web

SERVICIOS DE RED E INTERNET

HTTPS

- **HTTPS (Hypertext Transfer Protocol Secure)** es la versión segura de HTTP, diseñada para asegurar la transferencia de datos entre un cliente (como un navegador web) y un servidor. Aunque en sus inicios se utilizaba principalmente para proteger información sensible como contraseñas, tarjetas de crédito y datos privados, hoy en día su uso se ha extendido a casi todo tipo de tráfico en la web, gracias a la concienciación sobre la importancia de la seguridad en línea y la presión de gigantes como **Google**, que favorecen en sus resultados de búsqueda a los sitios que implementan HTTPS.

Capa de Aplicación en el Modelo TCP/IP

- El protocolo HTTPS trabaja en la **capa de aplicación**, que es la más alta en la pila de **5 capas de TCP/IP** (equivalente a las **7 capas del modelo OSI**). Esta capa es donde los usuarios interactúan directamente con las aplicaciones y servicios de la web, como navegadores y servidores, y donde se produce la comunicación entre estos elementos a través de protocolos como HTTP y HTTPS.

Creación de un Canal Seguro

- La característica distintiva de HTTPS es que establece un **canal seguro** de comunicación mediante **cifrado**. El nivel de seguridad lo definen dos actores:
 1. El cliente: El navegador que accede al sitio web.
 2. El servidor: El sitio web al que el navegador se conecta.
- Este canal cifrado protege la **integridad y confidencialidad** de los datos que se transmiten, haciendo difícil que puedan ser interceptados o alterados por terceros no autorizados. El **cifrado se negocia** en el inicio de la conexión utilizando un proceso conocido como **Handshake SSL/TLS**, donde cliente y servidor acuerdan cómo encriptar la información.

Evolución de SSL a TLS

- Inicialmente, HTTPS utilizaba un protocolo llamado **SSL (Secure Sockets Layer)** para establecer este canal seguro. Sin embargo, SSL ha sido **reemplazado** por **TLS (Transport Layer Security)**, que es más seguro y moderno. A día de hoy, cuando hablamos de HTTPS, estamos hablando en realidad de TLS, aunque en ocasiones se siga utilizando el término SSL por cuestiones históricas o de nomenclatura.

Puerto de HTTPS: 443 y 4433

- Mientras que HTTP estándar utiliza el puerto 80, HTTPS utiliza el puerto 443. Esto es importante, ya que el puerto 443 es el que está destinado a la comunicación cifrada en la mayoría de las aplicaciones web. Al utilizar este puerto, los navegadores y servidores están configurados para manejar de manera predeterminada conexiones seguras, lo que es vital para proteger los datos transmitidos.

Motivos de la Adopción Generalizada de HTTPS

- Seguridad Universal: HTTPS asegura que toda la comunicación entre el cliente y el servidor esté cifrada, protegiendo incluso información que en principio podría no parecer sensible. Esto evita posibles ataques como el "Man-in-the-Middle" (interceptación de tráfico).
- Reducción de Costes: Los certificados de seguridad SSL/TLS solían ser costosos, pero hoy en día, servicios como Let's Encrypt ofrecen certificados gratuitos, haciendo que implementar HTTPS sea accesible para casi cualquier sitio web. Mejor
- Posicionamiento en Google: Como parte de su política de promover un internet más seguro, Google ha comenzado a dar preferencia en sus resultados de búsqueda a los sitios web que utilizan HTTPS, penalizando aquellos que siguen utilizando HTTP. Además, los navegadores modernos, como Chrome, etiquetan los sitios sin HTTPS como "No seguros", lo que afecta negativamente a la confianza de los usuarios.

INTERCAMBIO DE CLAVES POR HTTPS (SSL/TLS)

Inicio de la conexión - Cliente envía "Client Hello"

- El proceso comienza cuando el cliente (por ejemplo, el navegador web) intenta conectarse a un servidor HTTPS. El cliente envía un mensaje llamado **"Client Hello"** al servidor. Este mensaje incluye:
 - La versión de TLS soportada por el cliente.
 - Una lista de **algoritmos de cifrado** soportados (ciphersuites).
 - Un número aleatorio (nonce) generado por el cliente que se usará más tarde para crear las claves de sesión.
 - Información de compresión y otras configuraciones de conexión.

INTERCAMBIO DE CLAVES POR HTTPS (SSL/TLS)

El servidor responde con "Server Hello"

- El servidor responde con un mensaje llamado "Server Hello". En este mensaje:
 - El servidor selecciona una versión de TLS y un algoritmo de cifrado de los propuestos por el cliente.
 - Se envía otro número aleatorio (nonce) generado por el servidor.
 - El servidor también envía su **certificado digital**, que contiene su **clave pública** y está firmado por una Autoridad de Certificación (CA). Este certificado permite que el cliente autentique al servidor.

INTERCAMBIO DE CLAVES POR HTTPS (SSL/TLS)

Verificación del certificado del servidor

- El cliente recibe el certificado y lo valida. Los pasos de validación son:
 - Comprobar que el certificado ha sido emitido por una CA confiable (según la lista de CAs instaladas en el cliente).
 - Verificar que el certificado no ha expirado y que es válido para el dominio al que el cliente intenta conectarse.
 - Asegurarse de que el certificado no ha sido revocado (mediante CRL o el protocolo OCSP).
 - Si el certificado es válido, el cliente sigue adelante con el proceso de intercambio de claves. Si no es válido, el cliente mostrará un aviso de seguridad.

INTERCAMBIO DE CLAVES POR HTTPS (SSL/TLS)

Intercambio de claves (Pre-Master Secret)

- Una vez validado el certificado del servidor, se realiza el **intercambio de claves**. Dependiendo del algoritmo de cifrado elegido, los métodos para el intercambio de claves pueden variar. Los dos métodos más comunes son:
 - **RSA** (criptoasimétrica): El cliente genera un número aleatorio llamado **Pre-Master Secret** y lo cifra usando la **clave pública** del servidor obtenida del certificado. Luego, el cliente envía el Pre-Master Secret cifrado al servidor. Solo el servidor, con su **clave privada**, puede descifrarlo.
 - **Intercambio de claves Diffie-Hellman**: En algunos casos, se utiliza el protocolo **Diffie-Hellman** (DH o ECDH, en su versión elíptica) para generar de forma segura una clave compartida entre el cliente y el servidor. En este caso, no se envía directamente el Pre-Master Secret, sino que tanto el cliente como el servidor usan sus propios valores aleatorios para calcular el secreto compartido.

INTERCAMBIO DE CLAVES POR HTTPS (SSL/TLS)

Generación de claves de sesión

- Una vez que el servidor ha descifrado el **Pre-Master Secret** o completado el intercambio Diffie-Hellman, tanto el cliente como el servidor usan este secreto compartido junto con los números aleatorios intercambiados en los pasos anteriores (de los mensajes Client Hello y Server Hello) para generar la **clave de sesión**. Esta clave de sesión se utiliza para cifrar toda la comunicación posterior utilizando **cifrado simétrico**, que es más eficiente que la criptografía asimétrica.

INTERCAMBIO DE CLAVES POR HTTPS (SSL/TLS)

Cliente envía "Finished"

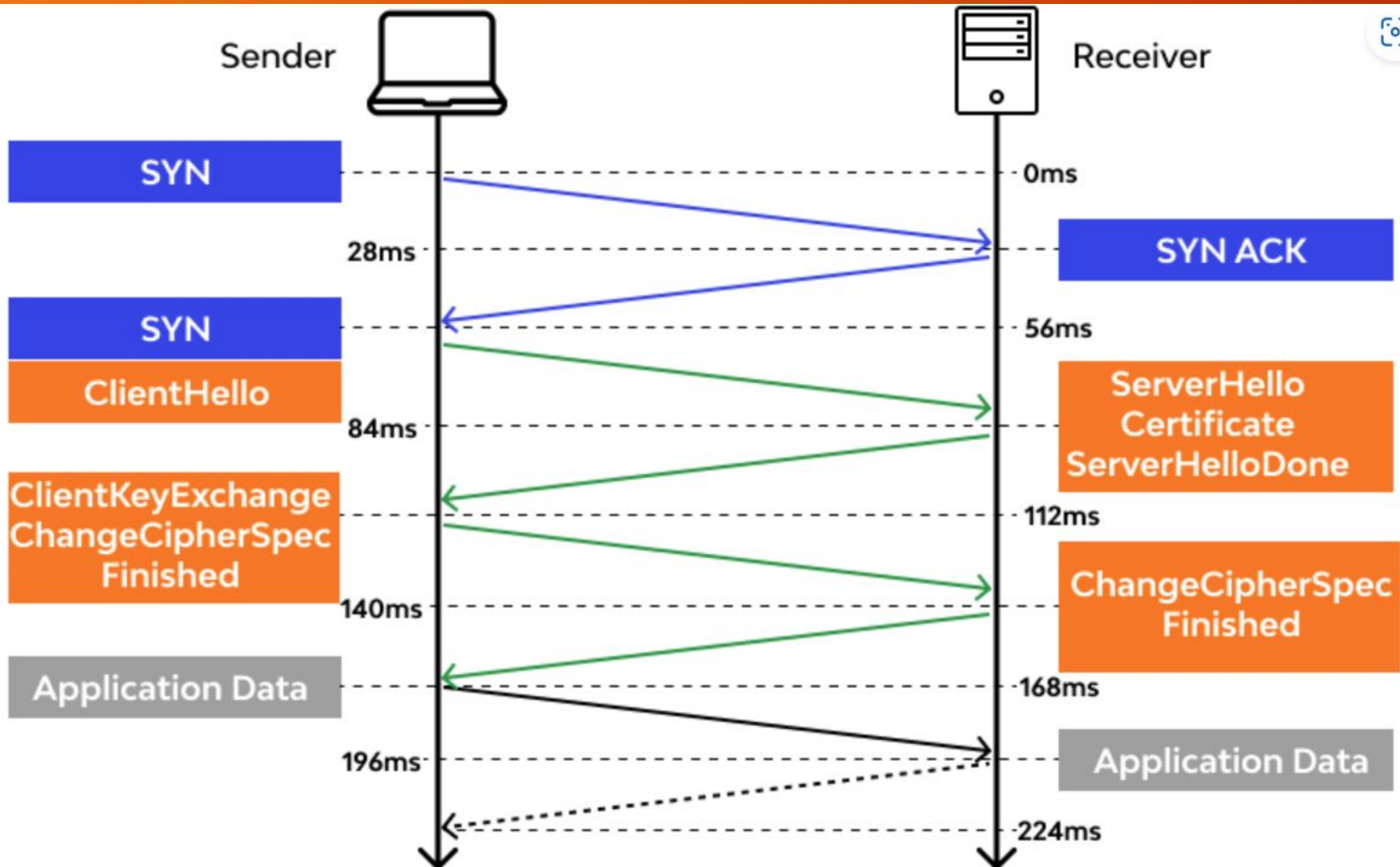
- El cliente ahora envía un mensaje cifrado llamado **"Finished"** al servidor usando la clave de sesión recién generada. Este mensaje incluye un **hash** de todos los mensajes intercambiados durante el establecimiento de la conexión, asegurando que ningún tercero ha alterado el proceso.

Servidor envía "Finished"

- El servidor también envía un mensaje cifrado **"Finished"** al cliente, indicando que ha completado el proceso de negociación y que está listo para comenzar la transmisión segura de datos.

Inicio de la comunicación cifrada

- En este punto, tanto el cliente como el servidor han establecido una clave de sesión segura y simétrica. Toda la comunicación posterior, incluyendo las peticiones y respuestas HTTP, estará cifrada con esta clave de sesión.



SSL/TLS: Seguridad en las Transacciones en Internet

- El protocolo SSL (Secure Socket Layer), que luego fue reemplazado por TLS (Transport Layer Security), es el principal medio utilizado para garantizar la **seguridad de las transacciones** en Internet. Ambos protocolos permiten la **comunicación segura** entre un usuario (cliente) y un servidor web, asegurando que los datos transmitidos no sean interceptados ni modificados por terceros.

SSL/TLS: Seguridad en las Transacciones en Internet

SSL (Secure Socket Layer)

- SSL fue desarrollado para asegurar que las transacciones realizadas en Internet fueran **privadas e íntegras**. Aunque ha sido reemplazado por TLS, los conceptos que introdujo siguen siendo esenciales para entender cómo funciona la seguridad en la web. SSL permitía:
- **Cifrado de datos:** La información que se intercambia entre el cliente y el servidor se cifra para que no pueda ser leída por terceros.
- **Autenticación de servidores:** Los clientes pueden verificar que están conectados al servidor correcto (generalmente a través de certificados digitales).
- **Integridad de mensajes:** Garantiza que los datos no se hayan modificado durante la transmisión.
- **Autenticación opcional del cliente:** A veces, también se puede verificar la identidad del cliente.

SSL/TLS: Seguridad en las Transacciones en Internet

TLS (Transport Layer Security)

- TLS es la versión mejorada y más segura de SSL. Utiliza **certificados X.509** y combina **criptografía asimétrica** y **simétrica** para proteger la comunicación. Los principales objetivos de TLS son:
 - **Confidencialidad:** Todos los datos transmitidos entre el cliente y el servidor están cifrados.
 - **Integridad:** Los mensajes se protegen contra la modificación o manipulación mediante el uso de **hashes**.
 - **Autenticación:** El cliente puede verificar que está hablando con el servidor correcto (autenticación del servidor), y opcionalmente, el servidor puede autenticar al cliente.

Criptografía en SSL/TLS

Cifrado Simétrico

- En el cifrado simétrico, se utiliza una sola clave para **cifrar y descifrar** la información. Este método es rápido y eficiente, pero presenta el inconveniente de que ambas partes (cliente y servidor) necesitan compartir la misma clave. El mayor desafío del cifrado simétrico es la **distribución de las claves**, ya que si se transmite por un canal inseguro, podría ser interceptada. Ejemplos de algoritmos de cifrado simétrico incluyen **AES (Advanced Encryption Standard)** y **DES (Data Encryption Standard)**.
- **Ventaja:** Muy rápido y eficiente en términos de procesamiento.
- **Desventaja:** La clave debe ser compartida, lo que introduce un riesgo si se transmite por un canal inseguro.

Criptografía en SSL/TLS

Cifrado Asimétrico

- El cifrado asimétrico utiliza un **par de claves**: una **clave pública** y una **clave privada**. La clave pública se puede compartir con cualquier persona y se utiliza para cifrar los mensajes, pero solo la clave privada (que se mantiene secreta) puede descifrar esos mensajes. A la inversa, si se firma algo con la clave privada, cualquier persona con la clave pública puede verificar la autenticidad del remitente.
- **Ventaja**: No es necesario compartir la clave privada, por lo que es mucho más seguro para el intercambio inicial de información.
- **Desventaja**: Es mucho más lento en comparación con el cifrado simétrico debido a la complejidad de los algoritmos.

Funcionamiento de SSL/TLS en la Práctica

- **Inicio de la Conexión Segura:**

- Cuando un cliente (por ejemplo, un navegador web) se conecta a un servidor usando HTTPS, el primer paso es un proceso llamado **handshake**.

- **Autenticación del Servidor:**

- El cliente verifica que el **certificado** presentado por el servidor es válido y ha sido emitido por una **Autoridad de Certificación (CA)** confiable.

- **Intercambio de Claves:**

- El cliente y el servidor colaboran para crear una clave compartida, que se utilizará en el cifrado simétrico para toda la sesión.

- **Cifrado de Datos con Clave Simétrica:**

- Una vez que el cliente y el servidor han establecido una clave compartida (clave de sesión), la usan para cifrar y descifrar todos los datos que se transmiten entre ellos

- **Integridad de Mensajes:**

- SSL/TLS también garantiza la integridad de los mensajes mediante el uso de **códigos de autenticación de mensajes (MAC)** o **hashes criptográficos**, como **SHA-256**. Esto asegura que los datos no hayan sido modificados en tránsito.

Seguridad en Servicios Web

- Cuando hablamos de **seguridad en servicios web**, existen una serie de **principios fundamentales** que deben ser garantizados para proteger la **información** y las **transacciones** que se realizan en línea. Estos principios se aplican tanto a la **infraestructura del servidor** como a las **comunicaciones entre el cliente y el servidor**, y son clave para mantener la **confianza** en los servicios que ofrecen las aplicaciones web.

Seguridad en Servicios Web

Confidencialidad La confidencialidad se refiere a la capacidad de mantener la información privada y evitar que sea accesible por personas o sistemas no autorizados. En el contexto de los servicios web, esto significa que los datos transmitidos entre el cliente y el servidor deben estar cifrados para que no puedan ser interceptados y leídos por terceros. SSL

Confidencialidad



Disponibilidad

La **disponibilidad** se refiere a la capacidad de acceder a la información y los servicios cuando se necesitan. Los sistemas y los servicios web deben estar disponibles para los usuarios y clientes autorizados en todo momento, evitando interrupciones que puedan comprometer el servicio. DDOS

Integridad

La **integridad** se refiere a la garantía de que los datos no han sido alterados o modificados durante la transmisión. En un servicio web, es fundamental asegurarse de que la información enviada entre el cliente y el servidor no ha sido manipulada por terceros. SSL

Seguridad en Servicios Web

Autenticación y Autorización

- La **autenticación** es el proceso mediante el cual se verifica la **identidad** de una persona o sistema que intenta acceder a un recurso web, mientras que la **autorización** es el proceso que determina qué acciones o recursos puede acceder una entidad autenticada.
- **Autenticación:** Puede ser **anónima** (sin que se verifique la identidad), pero normalmente implica el envío de **credenciales** como nombre de usuario y contraseña. Existen otros métodos más avanzados como:
 - **Autenticación de dos factores (2FA):** Implica un segundo paso de verificación, como un código enviado al teléfono móvil o una aplicación de autenticación.
 - **Autenticación de terceros:** Algunos sitios web permiten que los usuarios se autenticen usando servicios como **Google**, **Facebook** o **Microsoft**, donde el proceso de autenticación es gestionado por estos proveedores.
- **Autorización:** Una vez autenticado, el sistema web debe controlar los **niveles de acceso**. Esto se refiere a quién puede ver o modificar información, y qué recursos puede utilizar dentro del sistema. La autorización generalmente se gestiona mediante **roles** o **permisos** asignados a los usuarios en función de su identidad y credenciales.

Seguridad en Servicios Web

Métodos Comunes de Autenticación

- **Usuario y contraseña:** El método más simple, pero también uno de los menos seguros si no se acompaña de otras medidas como 2FA.
- **Autenticación basada en correo electrónico o teléfono:** El usuario confirma su identidad a través de un código enviado a su correo o teléfono.
- **Autenticación federada:** Utilizando servicios de terceros (por ejemplo, iniciar sesión con Google, Facebook, etc.). Este método permite simplificar el inicio de sesión para los usuarios, mientras que la seguridad se maneja mediante estos servicios confiables.

Importancia de la Seguridad en Servicios Web

- En un entorno en el que gran parte de la información sensible y las transacciones se realizan a través de la web (datos personales, bancarios, etc.), garantizar la **seguridad** en los servicios web es crucial para prevenir **fraudes, ataques de intermediarios** (man-in-the-middle), **robo de datos** y otros riesgos cibernéticos.

Importancia de la Seguridad en Servicios Web

- Además de los protocolos SSL/TLS y HTTPS, es importante implementar otros mecanismos adicionales de seguridad en las aplicaciones web, tales como:
- Firewalls de aplicaciones web (WAF).
- Protección contra ataques de fuerza bruta.
- Sistemas de detección de intrusos (IDS).
- Buenas prácticas de desarrollo seguro (como la validación de entradas de usuarios para evitar inyecciones SQL o ataques XSS).

PROVEEDORES WEB

- La mayoría de los principales **proveedores de hosting en España** hoy en día **incluyen certificados SSL gratuitos** en sus planes, lo que garantiza la seguridad mediante HTTPS sin costo adicional, una característica esencial para mantener la confidencialidad y autenticidad de los sitios web. Esto solía ser un servicio premium, pero con el auge de la importancia de la seguridad y la presión de motores de búsqueda como Google, que priorizan los sitios con HTTPS, ahora es común encontrarlo en casi todos los planes de alojamiento.

Proveedores destacados y políticas sobre HTTPS:

- **Hostinger:** Ofrece **SSL ilimitado gratuito** en todos sus planes, junto con **migración gratuita** de sitios web y ancho de banda ilimitado. Es conocido por ser accesible, con precios que comienzan en **1,49 €/mes**, lo que lo hace ideal para proyectos pequeños y medianos
- **Raiola Networks:** Incluye **SSL gratuito** en sus planes y se destaca por su enfoque en **seguridad y rendimiento**. Además, ofrece protección contra ataques DDoS y copias de seguridad diarias, lo que lo convierte en una opción robusta para sitios que requieren alta disponibilidad
- **IONOS:** Proporciona **certificados SSL gratuitos** en todos sus planes, además de copias de seguridad diarias. Su precio comienza en **3 €/mes**, y está bien posicionado para proyectos de tamaño pequeño a mediano
- **Hostalia:** Esta empresa ofrece **certificado SSL gratuito** en la mayoría de sus planes. Destaca por su **soporte técnico 24/7** y la facilidad de uso de sus paneles de control. Además, Hostalia incluye herramientas de seguridad para proteger los sitios web de posibles ataques
- **Webempresa:** Un proveedor que pone un fuerte énfasis en la **seguridad**, incluye **SSL gratuito** en todos sus planes. Además, ofrece **soporte especializado en WordPress**, lo que lo convierte en una opción atractiva para los sitios basados en este CMS. Los planes empiezan desde **2,47 €/mes**
- **GoDaddy:** Aunque conocido por su facilidad de uso y amplias ofertas de productos, **GoDaddy** incluye **SSL gratuito** en algunos de sus planes de hosting. Sin embargo, en los planes más básicos puede ser necesario adquirirlo por separado. GoDaddy también ofrece **migración de sitios y herramientas de seguridad** para mejorar la protección del sitio

Aplicaciones web

- Una **aplicación web** es un software que se utiliza a través de un navegador web, utilizando protocolos como **HTTP** o **HTTPS**. Estos permiten que los usuarios accedan a funcionalidades o datos almacenados en un servidor remoto. A diferencia de las aplicaciones de escritorio, las aplicaciones web no requieren instalación en el dispositivo del usuario. Algunos ejemplos comunes incluyen aplicaciones bancarias en línea, plataformas de gestión de contenido (CMS), o herramientas de productividad como Google Docs.
- El uso creciente de **aplicaciones web**, **web services** y **API** ha llevado a un aumento de las demandas de **seguridad** en los servidores que los soportan. Estos servicios son fundamentales para la interoperabilidad y el acceso eficiente a los datos a través de redes.

APIs

- Las **API (Interfaz de Programación de Aplicaciones)**, por su parte, son interfaces que permiten que diferentes aplicaciones se comuniquen entre sí. Están diseñadas para **procesar solicitudes y responder** con datos, permitiendo la integración entre sistemas. Por ejemplo, una API puede ser utilizada por una aplicación móvil para acceder a los servicios de una plataforma en la nube.

APIs

Las APIs aportan ventajas clave, como:

- **Flexibilidad:** Permiten modificar partes del sistema sin afectar a todo el conjunto.
- **Portabilidad:** Las APIs son agnósticas respecto a las plataformas y lenguajes de programación, lo que facilita su implementación en diferentes entornos.
- **Rápidas actualizaciones:** Se pueden actualizar partes específicas del sistema sin interrumpir su funcionamiento global, lo que agiliza el mantenimiento y la mejora continua.

Ejecución de scripts en el servidor

- Desde los primeros días de la web estática, los desarrolladores han buscado formas de crear **páginas web dinámicas** que puedan responder a las peticiones de los usuarios de manera personalizada. Este concepto es fundamental en el **desarrollo back-end**, que se refiere a la parte del desarrollo web que se ejecuta en el servidor y se encarga de gestionar las **bases de datos**, **lógica de negocio**, y la **respuesta a las solicitudes** del usuario.

Ejecución de scripts en el servidor

Tecnologías iniciales: CGI

- Inicialmente, se utilizaba CGI (Common Gateway Interface) para permitir la interacción entre los servidores web y los programas que generaban contenido dinámico. CGI era una forma de "pasarela" entre el servidor web y los programas externos escritos en lenguajes como Perl, Python o C. Cuando un cliente hacía una petición al servidor, CGI actuaba como intermediario y ejecutaba el script necesario para generar el contenido solicitado.
- **Desventaja:** Aunque CGI abrió la puerta a las primeras aplicaciones web dinámicas, tenía problemas de rendimiento. Cada vez que el servidor recibía una solicitud, se creaba un nuevo proceso, lo que generaba una sobrecarga significativa en el servidor, especialmente bajo alta demanda.

Ejecución de scripts en el servidor

Evolución hacia lenguajes integrados como PHP

- Con el tiempo, se introdujeron lenguajes de programación **nativos del servidor**, como **PHP**, que mejoraron la eficiencia al permitir la ejecución de scripts directamente dentro del servidor web, sin necesidad de procesos externos. PHP se integra dentro del servidor y puede procesar solicitudes de forma más rápida y eficiente que CGI. Otros lenguajes como **Python**, **Ruby** y **Node.js** también comenzaron a ser populares en el desarrollo back-end.
- **Ventaja:** PHP y lenguajes similares permiten una integración más fluida y un rendimiento mucho mejor, ya que el servidor puede manejar múltiples solicitudes sin crear un nuevo proceso para cada una.

Ejecución de scripts en el servidor

Servidores de aplicaciones

- Para aplicaciones web más complejas, se introdujo el concepto de **servidores de aplicaciones**. Estos son programas especializados que se ejecutan en el servidor y ayudan al **servidor web** a procesar las páginas que contienen **scripts del lado del servidor**. En este modelo, cuando el servidor web recibe una solicitud para una página dinámica, pasa el procesamiento de esa página al servidor de aplicaciones, que luego devuelve la página procesada para que el servidor web la envíe al navegador.

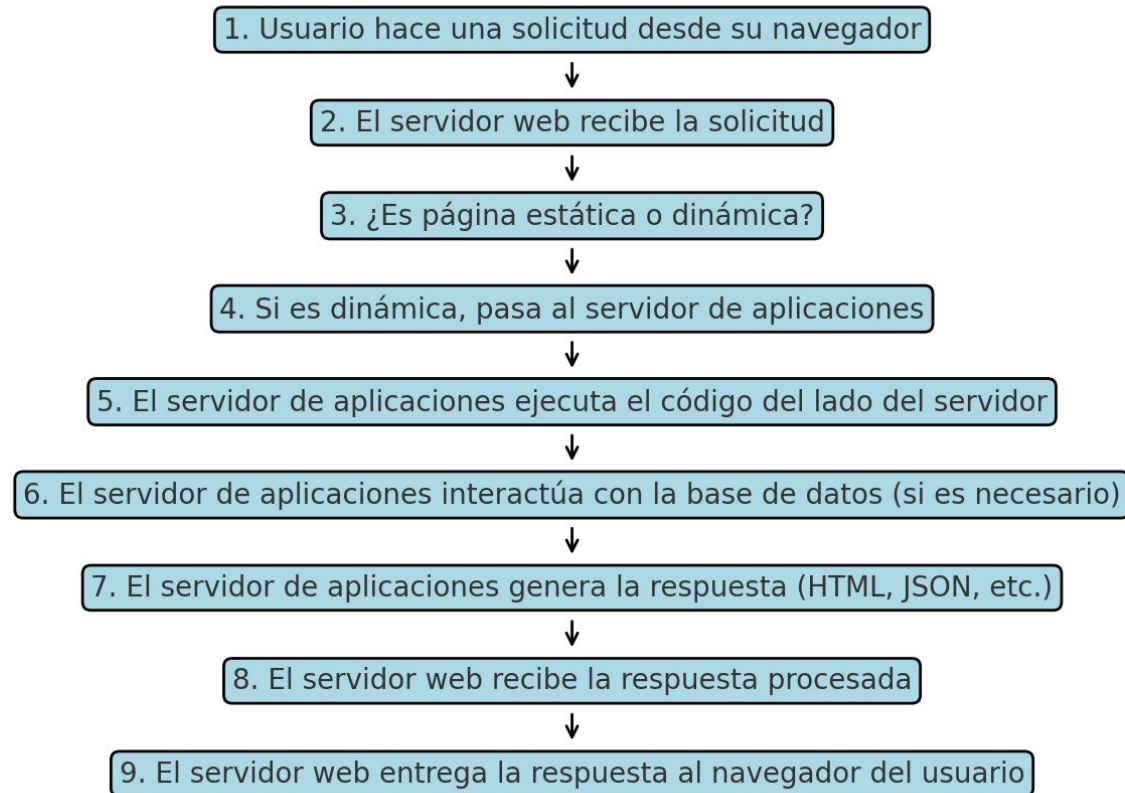
Ejemplos de servidores de aplicaciones

- **Apache Tomcat:** Usado principalmente para ejecutar aplicaciones Java.
- **Node.js:** Popular para aplicaciones basadas en JavaScript del lado del servidor.
- **Django:** Un framework de Python que integra tanto el servidor web como el procesamiento de aplicaciones.



Arquitectura de procesamiento

Diagrama de Flujo: Arquitectura de Procesamiento de Solicitudes Web



Ejecución de scripts en el cliente

- En una **aplicación web**, no toda la lógica y procesamiento se realiza en el servidor. Una parte importante de las operaciones puede ser ejecutada en el **lado del cliente**, es decir, directamente en el navegador del usuario. Esto se realiza a través de lenguajes de programación como **JavaScript**, que permite a los navegadores web ejecutar código sin tener que comunicarse constantemente con el servidor.

Ejecución de scripts en el cliente

Validaciones en el cliente

- Uno de los usos más comunes de los scripts del lado del cliente es la **validación de formularios**. Antes de enviar una solicitud al servidor, se pueden validar ciertos campos (como asegurarse de que un correo electrónico tenga el formato correcto o que no falten campos obligatorios). Esto reduce la cantidad de interacciones con el servidor, mejorando la **eficiencia y velocidad** de la aplicación.

Ejecución de scripts en el cliente

Lógica de negocio en el cliente

- Además de las validaciones, algunas partes de la **lógica de negocio** también pueden ejecutarse en el lado del cliente. Esto incluye funciones que generan **interactividad y dinamismo** en la página web sin necesidad de recargarla por completo.

Ejecución de scripts en el cliente

Interactividad y rendimiento

- El uso de JavaScript en el cliente permite crear aplicaciones más interactivas y mejorar la experiencia del usuario. Tecnologías como React, o Angular son frameworks de JavaScript que permiten construir aplicaciones de una sola página (SPA), donde todo el contenido dinámico se actualiza sin la necesidad de hacer múltiples solicitudes al servidor.

Ejecución de scripts en el cliente

Seguridad

- Aunque ejecutar scripts en el lado del cliente tiene muchos beneficios, también plantea ciertos **riesgos de seguridad**. Los usuarios malintencionados pueden modificar o manipular los scripts ejecutados en su navegador. Por este motivo, es crucial que las validaciones y la lógica de negocio **importante** también se realicen en el servidor, para evitar fraudes o manipulación de datos.

Servidores Web

- Un **servidor web** es un software que se ejecuta en un servidor físico o virtual y su función principal es **entregar contenido web** a los clientes que lo soliciten a través de sus navegadores. Este contenido puede incluir **páginas HTML, imágenes, videos** y otros tipos de archivos. El servidor web se encarga de **procesar las solicitudes** que recibe de los navegadores mediante el protocolo **HTTP** (o **HTTPS** si se trata de una conexión segura), y luego envía la respuesta con los datos solicitados.

Servidores Web

Ejemplos de servidores web:

- **Apache HTTP Server:** Uno de los servidores web más utilizados en el mundo, conocido por su robustez y flexibilidad.
- **Microsoft IIS:** El servidor web de Microsoft, diseñado para integrarse perfectamente con el entorno Windows.
- **Nginx:** Un servidor web ligero y de alto rendimiento, conocido por su capacidad de manejar múltiples conexiones simultáneas.

Características de un servidor web

Soporte de múltiples protocolos:

- **HTTP y HTTPS:** Los protocolos fundamentales para la transmisión de páginas web. HTTPS proporciona una capa adicional de seguridad mediante cifrado.
- **FTP:** Para la transferencia de archivos entre el cliente y el servidor.
- **SSH:** Protocolo utilizado para el acceso seguro y la administración remota del servidor.

Características de un servidor web

- **Interfaz de administración web:**
- La mayoría de los servidores web modernos permiten la administración de su configuración a través de una **interfaz gráfica basada en la web**, lo que facilita la gestión de recursos, dominios y configuraciones sin necesidad de usar la línea de comandos.

Características de un servidor web

Soporte para bases de datos y scripting:

- **Bases de datos:** Integración con **MySQL**, **PostgreSQL** o **MariaDB** para almacenar y gestionar datos en aplicaciones web.
- **Scripting:** Los servidores web pueden ejecutar lenguajes de programación como **PHP**, **Python**, **Ruby** o **Node.js** para generar contenido dinámico en respuesta a las solicitudes del cliente.

Características de un servidor web

Seguridad:

- **Autenticación de usuarios:** Funcionalidades integradas para autenticar usuarios antes de permitir el acceso a ciertas partes del sitio.
- **Protección contra ataques:** Los servidores web suelen implementar herramientas para protegerse de ataques comunes, como Denegación de Servicio (DoS) o Inyección SQL.

Características de un servidor web

Alojamiento multilingüe y gestión de errores:

- **Idiomas:** Capacidad para servir sitios web en múltiples idiomas, dependiendo de la configuración del cliente.
- **Redirección de tráfico:** Si hay un error en el sitio principal, el servidor puede redirigir automáticamente el tráfico a una página de respaldo o alternativa.

Características de un servidor web

Gestión de recursos:

- **Cuotas de ancho de banda:** Limitar el ancho de banda para ciertos usuarios o dominios, lo que ayuda a evitar la sobrecarga del servidor.
- **Cuotas de espacio en disco:** Controlar el uso del espacio en disco de los usuarios, lo que es especialmente importante en entornos de alojamiento compartido.

Configuración de un servidor web: Parámetros de configuración comunes

Archivos de configuración:

- En servidores como **Apache**, el archivo principal de configuración es el **httpd.conf**, mientras que en **Nginx** se utiliza el archivo **nginx.conf**. Estos archivos contienen directivas que controlan cómo el servidor debe gestionar las solicitudes y recursos.
- Estas directivas suelen estar organizadas en **secciones** y usan sintaxis estructurada (a menudo basada en **XML** o similar) para definir el comportamiento del servidor.

Configuración de un servidor web:

Parámetros de configuración comunes

Directivas de configuración:

- Las **directivas** son comandos que le indican al servidor cómo manejar diferentes aspectos del servicio. Por ejemplo:
 - **Listen**: Define el puerto en el que el servidor debe escuchar (por ejemplo, 80 para HTTP o 443 para HTTPS).
 - **DocumentRoot**: Indica el directorio donde se encuentran los archivos que se servirán a los clientes.
 - **ErrorLog**: Define el archivo donde se almacenarán los mensajes de error del servidor.

Configuración de un servidor web: Parámetros de configuración comunes

Estructuración y organización del archivo de configuración:

- En caso de que el archivo de configuración crezca en tamaño y complejidad, es una buena práctica **dividirlo en secciones** o usar **archivos de inclusión**
- **Comentarios:** Incluir comentarios en el archivo de configuración ayuda a otros administradores a entender rápidamente lo que hace cada sección.

Configuración de un servidor web:

Parámetros de configuración comunes

Reinicio del servidor:

- En muchas situaciones, cuando se modifica la configuración, es necesario **reiniciar** el servidor o al menos **recargar** el servicio para que los cambios surtan efecto. Esto es común tanto en servidores basados en **Windows** como en **Linux**.

Copias de seguridad:

- Dado que los archivos de configuración son críticos para el funcionamiento del servidor, es importante hacer **copias de seguridad** de estos archivos antes de realizar cambios importantes. De esta manera, se puede restaurar rápidamente una configuración anterior en caso de que algo salga mal.

Arranque y Parada de un Servidor Web

- El **arranque** de un servidor web es el proceso mediante el cual se inicia el servidor y se ponen en funcionamiento los servicios web, lo que permite que los usuarios comiencen a acceder al contenido o las aplicaciones alojadas. Durante este proceso, el servidor web carga su configuración, se conecta a las bases de datos y establece las conexiones necesarias para poder responder a las solicitudes HTTP/HTTPS.
- Por otro lado, la **parada** de un servidor web implica detener todos los servicios y liberar los recursos asociados (como CPU, memoria y conexiones). Este proceso es esencial cuando es necesario realizar **mantenimiento**, actualizaciones o reparaciones en el sistema. Es recomendable tener una arquitectura con **redundancia de servidores** y **balanceo de carga** para garantizar que el servicio web no se vea interrumpido durante las tareas de mantenimiento. Esta configuración asegura que si un servidor se detiene, otro puede tomar su lugar sin afectar la disponibilidad del servicio.

Servidores Virtuales

- Un **servidor virtual** es una máquina virtual que se ejecuta dentro de un servidor físico. Un único servidor físico puede tener varios servidores virtuales, que actúan de manera independiente, como si fueran servidores físicos separados.
- Los **servidores virtuales** permiten maximizar el uso de los recursos de hardware, compartiendo CPU, memoria y espacio en disco, lo que ahorra costes y facilita la administración. Cada servidor virtual puede ejecutar su propio sistema operativo y estar configurado para diferentes propósitos.

Servidores Virtuales

Características de los servidores virtuales:

- **Ambivalentes:** Pueden actuar como un servidor físico independiente o compartir recursos con otros servidores virtuales en el mismo hardware.
- **Escalables:** Los recursos como CPU y memoria se pueden ajustar según las necesidades del servidor virtual, aumentando o disminuyendo de forma flexible.
- **Seguros:** Solo se puede acceder a ellos de manera autorizada, garantizando la seguridad de los datos y el acceso.
- **Eficientes:** Al compartir recursos, los servidores virtuales permiten un uso más eficiente de la energía y el almacenamiento.
- **Fiables:** Los servidores virtuales suelen tener sistemas de respaldo y recuperación, lo que garantiza que los datos se puedan recuperar en caso de fallo.
- **Flexibles:** Pueden instalar y ejecutar una amplia variedad de aplicaciones, lo que facilita su administración y mantenimiento.

Servidores Virtuales

- VMware vSphere/ESXi
- Microsoft Hyper-V
- Proxmox VE

Servidores Virtuales

Tipos de alojamiento virtual:

- Existen tres tipos principales de alojamiento virtual:
 1. **Basado en IP:** Cada sitio web tiene su propia dirección IP.
 2. **Basado en nombres:** Varios sitios web comparten una misma dirección IP, pero se identifican mediante su nombre de dominio.
 3. **Basado en puertos:** Diferentes sitios web se alojan en el mismo servidor y dirección IP, pero usan diferentes números de puerto para diferenciarlos.

Servidor Web en Linux y Windows

Servidor Web en Linux: Apache y XAMPP

- En Linux, uno de los servidores web más utilizados es **Apache HTTP Server**, ampliamente conocido por su estabilidad y flexibilidad. Apache se puede instalar como un componente independiente o como parte de un paquete integrado, como **XAMPP**.

Servidor Web en Linux y Windows

Servidor Web en Windows: IIS (Internet Information Services)

- En Windows, el servidor web por excelencia es Internet Information Services (IIS), que es parte del sistema operativo Windows Server, aunque también puede instalarse en versiones estándar de Windows como una característica opcional.

Alternativas comunes: Nginx

- Nginx es otro servidor web ampliamente utilizado en Linux, Windows, y macOS. Aunque comenzó como un servidor web optimizado para manejar grandes cantidades de conexiones concurrentes, hoy en día se utiliza también como **proxy inverso** y servidor de aplicaciones.

NAVEGADORES DE INTERNET



Monitorización, Logs y Elaboración de Pruebas en Servidores Web

- La monitorización, el análisis de logs y la elaboración de pruebas son elementos esenciales para asegurar el rendimiento, la estabilidad y la seguridad de un servidor web. Estos procesos ayudan a identificar problemas de rendimiento, errores de configuración y vulnerabilidades que podrían comprometer el correcto funcionamiento del sistema o la seguridad de la aplicación.

Monitorización, Logs y Elaboración de Pruebas en Servidores Web

Pruebas de Carga

- Las pruebas de carga son esenciales para evaluar cómo un servidor web maneja **grandes volúmenes de tráfico**. Estas pruebas envían múltiples solicitudes simultáneamente al servidor para analizar su rendimiento bajo condiciones de estrés. Su objetivo es determinar la capacidad del servidor para gestionar el tráfico, detectar posibles cuellos de botella y asegurar que el sitio web o la aplicación es escalable y estable bajo demanda.
- **Características evaluadas en pruebas de carga:**
- **Rendimiento:** ¿El servidor mantiene tiempos de respuesta aceptables bajo carga?
- **Capacidad de respuesta:** ¿Cómo afecta un mayor número de solicitudes a los tiempos de respuesta?
- **Estabilidad:** ¿El servidor sigue funcionando correctamente cuando se somete a altos volúmenes de tráfico?
- **Escalabilidad:** ¿Puede el servidor manejar un aumento en la cantidad de usuarios sin degradar significativamente el rendimiento?

Monitorización, Logs y Elaboración de Pruebas en Servidores Web

Pruebas de Seguridad

- Las pruebas de seguridad son esenciales para identificar posibles vulnerabilidades en el servidor web. Estas pruebas se pueden realizar utilizando software especializado que simula ataques reales y analiza cómo responde el servidor ante tales amenazas.
- Tipos de pruebas de seguridad:
- Pruebas manuales: Un equipo especializado realiza pruebas de seguridad de manera manual, explorando diferentes vectores de ataque.
- Pruebas automatizadas: Se utilizan herramientas que simulan ataques automáticos para identificar vulnerabilidades, como inyecciones SQL, ataques de fuerza bruta, y cross-site scripting (XSS).

Monitorización, Logs y Elaboración de Pruebas en Servidores Web

Herramientas para pruebas de seguridad:

- **OWASP ZAP:** Herramienta de código abierto que permite analizar aplicaciones web y detectar vulnerabilidades.
- **Nessus:** Un escáner de vulnerabilidades que identifica fallos de seguridad en servidores y redes.
- **Burp Suite:** Herramienta que permite realizar pruebas de seguridad en aplicaciones web, como la identificación de problemas de autenticación o inyecciones de código.
- **Nikto:** Escáner web que verifica la existencia de problemas comunes de seguridad en servidores web.

Monitorización, Logs y Elaboración de Pruebas en Servidores Web

Formas de pruebas de seguridad:

- **Pruebas activas:** Implican la utilización de herramientas que interactúan directamente con el servidor, buscando explotar vulnerabilidades conocidas o no conocidas.
- **Pruebas pasivas:** Estas pruebas se realizan sin interactuar directamente con el servidor, recopilando información sobre su configuración y posibles vulnerabilidades sin intentar explotarlas.
- **Pruebas de seguridad internas y externas:**
 - **Internas:** Se realizan dentro de la red local, simulando ataques desde un entorno controlado y desde usuarios que ya tienen acceso a la red interna.
 - **Externas:** Se llevan a cabo desde una red externa, simulando ataques que provienen de fuera de la organización, como si se tratara de un atacante remoto.

Monitorización y Logs

La **monitorización** continua de un servidor web es crucial para detectar problemas de rendimiento y seguridad antes de que se conviertan en fallos graves. Los **logs** proporcionan información detallada sobre las operaciones del servidor, incluidas las solicitudes que recibe, las respuestas que genera y cualquier error que ocurra.

- **Elementos clave en la monitorización:**
 - **Uso de recursos:** CPU, memoria, ancho de banda y almacenamiento.
 - **Tiempos de respuesta:** Cuánto tiempo toma el servidor en procesar solicitudes.
 - **Errores de aplicación:** Análisis de errores registrados en los logs (como códigos de error HTTP 500).
 - **Anomalías de tráfico:** Detección de posibles patrones de tráfico inusuales que podrían indicar ataques o problemas de rendimiento.

Monitorización y Logs

Logs y su análisis:

- Los **logs** son archivos generados por el servidor web que contienen registros de todas las actividades del servidor, incluidas las solicitudes de los clientes, los tiempos de respuesta y los errores.

Tipos de logs:

- **Logs de acceso:** Detallan todas las solicitudes que recibe el servidor, junto con la respuesta proporcionada y el tiempo que tomó procesar cada solicitud.
- **Logs de error:** Registran cualquier error que ocurra mientras el servidor está en funcionamiento, como fallos de conexión o errores de aplicaciones.